Entrega 2 – Teoria da Computação e Linguagens Formais

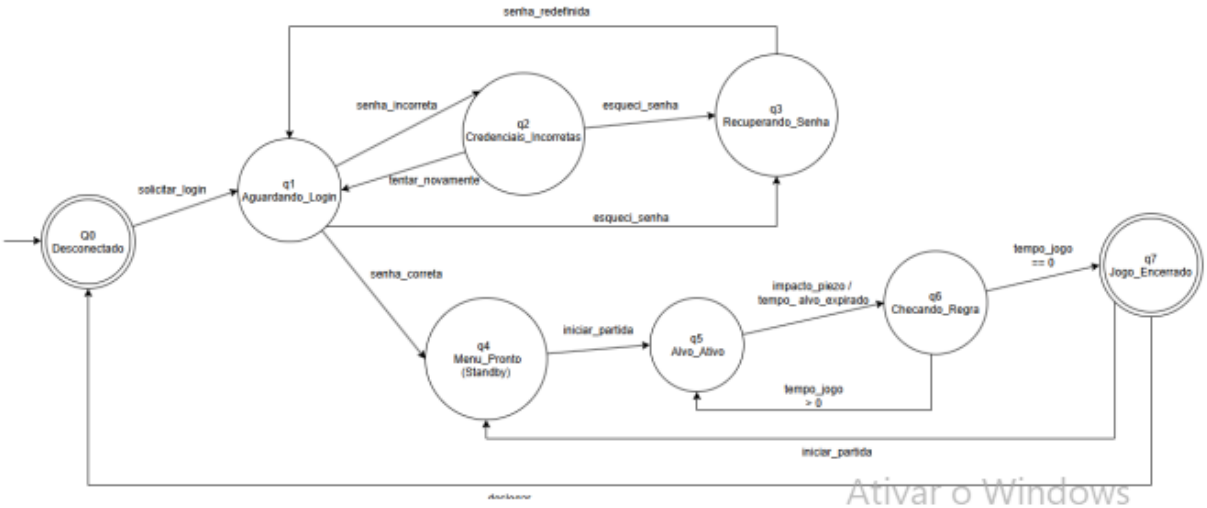
Grupo 5

Lucas Oliveira da Silva	RA:	24025988
Matheus Rossaneze Ramos Silva	RA:	24026354
Pedro Schaurich Maia	RA:	24026011
Phelipe Antonio de Souza	RA:	24026370

Introdução

Este documento é a entrega 2 da disciplina de Teoria da Computação e Linguagens Formais. Ele é uma atualização da entrega 1 então parte do conteúdo desta entrega é diretamente baseado na primeira, nosso projeto não sofreu grandes mudanças durante este intervalo.

1 – Atualização da máquina de Estados



Ativar o Windows

Sobre a máquina de estados ela não sofreu mudanças drásticas, durante o decorre do nosso processo de desenvolvimento do PI nós nos mantivermos firmes na maioria das decisões alterando principalmente questão físicas do Arduíno, mas poucas mudanças aplicamos no sistema em si.

2- Descrição dos eventos

Os eventos são os acontecimentos ou estímulos que provocam as transições de um estado para o outro.

- solicitar_login: Ação de abrir a tela de autenticação.
- senha_correta: Confirmação de credenciais válidas no sistema.
- senha_incorreta: Detecção de login ou senha inválidos.
- tentar_novamente: Ação de retornar ao formulário de acesso após um erro de login. esqueci_senha: Clique na opção de recuperação de senha.
- senha_redefinida: Conclusão bem-sucedida do envio/atualização de senha.
- iniciar_partida: Clique no botão do dashboard para dar início ao jogo.
- impacto_piezo: Sinal recebido pelo sensor piezoelétrico ao detectar a batida da bola no alvo. tempo_alvo_expirado: Término do tempo limite de um alvo individual sem impacto.
- tempo_jogo > 0: Condição que indica existência de tempo total de jogo restante (continua a partida).
- tempo_jogo == 0: Condição de encerramento do tempo total da partida.

3 – Regras de funcionamento

As regras de funcionamento descrevem as restrições lógicas e temporais que regem a execução segura do Alvo Inteligente. Na teoria de verificação formal de sistemas e Model Checking, tais propriedades dividem-se em duas categorias essenciais: Liveness que garantem que algo bom eventualmente acontecerá sem limite de tempo determinado e Safety que garantem que coisas ruins ao sistema não devam acontecer.

Abaixo são especificadas as cinco regras operacionais primárias do sistema:

RF-01: Exclusão Mútua de Partida (Safety)

Nenhuma partida pode ser iniciada se o sistema estiver deslogado (q_0) ou em estado de erro de credenciais (q_2). O acesso aos estados de jogo (q_5, q_6, q_7) exige obrigatoriamente a passagem prévia pela validação de senha (q_4).

RF-02: Integridade da Pontuação (Safety)

A pontuação só pode ser incrementada se a leitura analógica do sensor piezoelétrico ultrapassar a tensão limiar (V_lido ≥ V_ref) E o tempo individual do alvo for estritamente positivo (t_alvo > 0). Se o tempo expirar (t_alvo = 0), a tentativa é registrada como erro sem incremento de pontos.

RF-03: Sincronismo de Alvo e Hardware (Safety)

Enquanto um alvo estiver ativo (q_5), exatamente um dos 5 anéis de LEDs deve estar iluminado via registrador 74HC595 e o display TM1637 deve executar a contagem regressiva. Nenhum novo alvo pode ser sorteado enquanto o alvo corrente estiver em processamento.

RF-04: Parada Determinística por Tempo (Liveness)

Se o tempo total de partida atingir zero (tempo_jogo == 0), o autômato transita obrigatoriamente do estado de checagem (q_6) para o estado final de aceitação (q_7 - Jogo_Encerrado), interrompendo imediatamente a captura de sinais do piezo e travando novas pontuações.

RF-05: Recuperação e Tolerância a Erros (Liveness)

Na ocorrência de credenciais incorretas (q_2), o sistema deve permitir o retorno ao formulário (q_1) ou o redirecionamento ao fluxo de recuperação de senha (q_3). O sistema nunca deve entrar em estado de travamento indefinido (deadlock).

4 – Modelo Computacional do Projeto

Na Teoria da Computação, o poder expressivo dos modelos computacionais é categorizado pela Hierarquia de Chomsky. Para o sistema Alvo Inteligente Gamificado para Tênis, o modelo que melhor representa o sistema com exatidão e eficiência é o Autômato Finito Determinístico (AFD / FSM).

Os autômatos finitos correspondem ao modelo reconhecedor de menor complexidade computacional, caracterizado pela inexistência de memória auxiliar externa. Eles operam estritamente sobre um conjunto finito de estados (Q) e transitam de forma determinística com base nos eventos recebidos (Σ). No nosso projeto, o comportamento do sistema é inteiramente definido por estados estáticos discretos (como Desconectado, Aguardando_Login, Alvo_Ativo e Jogo_Encerrado) e transições guiadas por sinais de sensores e cronômetros. O sistema não necessita armazenar histórico ilimitado ou estruturas aninhadas diferentemente de um autômato de pilha, tornando o AFD o modelo formal perfeito, suficiente e matematicamente elegante.

5 – Análise de Decidibilidade

A decidibilidade é um dos principais conceitos da Teoria da Computação. Um **Problema de Decisão** é aquele cuja resposta é estritamente binária (SIM ou NÃO; True ou False; 0 ou 1). Um problema é considerado **Decidível** quando existe um algoritmo que recebe qualquer entrada válida, executa uma quantidade finita de passos, sempre termina (não fica executando infinitamente) e fornece a resposta correta.

No sistema do Alvo Inteligente, quatro avaliações fundamentais atuam como problemas de decisão decidíveis:

Problema de Decisão	Entrada do Algoritmo	Resposta Possível	Classificação / Garantia de Término
PD-01: Credenciais Válidas?	String de usuário e hash de senha fornecidos no formulário.	SIM (q_4) / NÃO (q_2)	Decidível. A busca na tabela de hashes é finita e sempre termina.
PD-02: Impacto Válido no Piezo?	Leitura analógica do pino do Arduino (0 a 1023).	SIM ($V \geq V_ref$) / NÃO	Decidível. Comparação de dois valores inteiros com resposta imediata.
PD-03: Tempo Total Expirado?	Registrador de tempo total da partida (cronômetro).	SIM ($t \leq 0$) / NÃO ($t > 0$)	Decidível. Subtração de clock finita e determinística.
PD-04: Cadeia de Eventos Aceita?	Sequência w de eventos sobre o alfabeto Σ.	SIM ($w \in L$) / NÃO ($w \notin L$)	Decidível. O autômato finito sempre consome a cadeia e para em estado F.

Conclusão

Aplicando os conceitos ensinados em sala de aula nós conseguimos desenvolver um sistema robusto e antecipamos diversos comportamentos que podem ocorrer durante a execução do sistema. Com o modelo de máquina de estados conseguimos criar um modelo visual para identificar com facilidade o fluxo das entradas no nosso sistema. Com a descrição dos eventos e regras de funcionamento conseguimos estruturar os comportamentos esperados assim criando o caminho para codar um sistema coeso. O modelo computacional do projeto e análise de decidibilidade fornecem uma documentação de referência tanto para nossa própria equipe quanto para os professores avaliadores compreenderem nosso projeto.